Write the Conveyor Program Yourself

A to Z in TIA Portal V21, Revision 3.2 (13 October 2026): new project, hardware, tags, data blocks, ladder logic, Modbus, HMI, download and test.
Every name, address, network and value is taken from the old project "latest plc", exported through TIA Portal itself.

New: read the companion guide "Understand the Conveyor Program" (Conveyor_Program_Explained.pdf, same folder). This guide says **what to click**; the companion says **what each part means, why it is there** (tags, data blocks, every network) and how to design a program from a problem statement. Its first page shows which part to read with which part of this guide.

What changed in Revision 3.2. Every item is now marked **needed** or **not needed**, so you do not spend time on things the machine never uses: an overview in part 0, a "Needed?" column in every tag, data block and HMI table, a mark on every FC_AUTO and FC_TCP_IP network, and a short 3-network version of FC_AUTO (part 8, way A) that behaves exactly like the original.

What changed in Revision 3.1. No more shortened rows. Revision 3 squeezed several items into one table row (for example "DI_SPARE I0.6, DI_SPARE I0.7 ... one row each"), which was confusing. Now **every tag, every data block row, every MOVE row and every lamp has its own row** with its own name, address and position: parts 2, 3, 8b, 10.2 and 10.6.

What changed in Revision 3. On 12 October 2026 the old project was opened in TIA Portal (a copy) and every block was exported by TIA itself. Revisions 1 and 2 were based on decoding the raw project file. Corrected now:
1. Part 1: the **clock memory byte** is switched ON at address 0 in the old project.
2. Part 2: the tags are in **three tag tables** (DI, DO, Default tag table), with spare tags.
3. Part 3: **DB_TCP_IP is optimized**; only the other four DBs are non-optimized.
4. Parts 4 - 8b: every network has its **real title** and **exact layout**. FC_MANUAL has the reverse row on top. **FC_AUTO is now the real six networks** (Revision 2 showed a shortened version). FC_TCP_IP: real row order, MB_SERVER **V5.3**.
5. Part 10 (HMI) is now from the HMI export: template **MAIN**, screens **AUTO, MANUAL, ALARMS, INPUT STATUS, OUTPUT STATUS** with their numbers, three HMI tag tables, buttons with **SetBitWhileKeyPressed**, the green / red status field, red / green I/O lamps, and the BACK button that points to its own screen.
6. New part 12: the red-button reverse jog you asked for. It is **not** in the old program and is added as a separate, optional step.

0. How to use this guide

- Work through the parts in order. Each part ends with something you can check.
- Type names **exactly** as shown, including spaces and capital letters. The HMI and Modbus depend on the names and addresses.
- The PLC part (parts 1 - 9) is exact: it was exported from the old project by TIA Portal itself (12 October 2026).
- The HMI part (part 10) is exact too, from the HMI export of 12 October 2026, except the alarm views and discrete alarms, which the export cannot include (marked CHECK).
- I could not run TIA Portal myself, so a menu name may differ by a word or two in your V21 installation. The position in the menus is the same.

Part	You build	About
1	New project and PLC hardware	15 min

Part	You build	About
2	Three PLC tag tables (inputs, outputs, flags)	20 min
3	Five data blocks	30 min
4 - 8b	Six program blocks (FC) in ladder	2 hours
9	Main block OB1, compile	10 min
10	HMI panel	2 hours
11	Download and test	1 hour
12	Optional, new: red-button reverse jog (not in the old program)	30 min

What is needed, and what is only for an identical copy

You do not have to build everything. This guide marks every item:

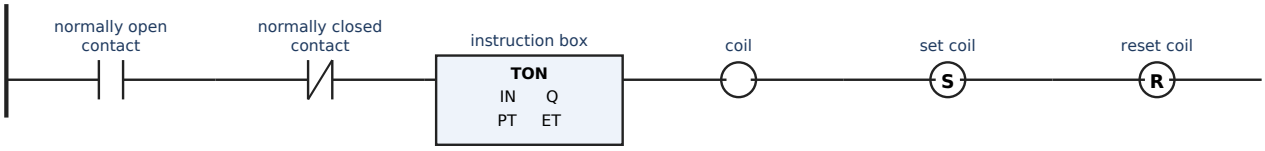
Mark	Meaning
YES / NEEDED	The machine needs it. Without it the program does not compile or the machine does not work.
USEFUL	The machine runs without it, but the operator loses information (a status text, an alarm text).
NO / NOT needed	Nothing uses it. Build it only if you want your project to look exactly like the old one.

Minimum build = only the YES items (and the USEFUL ones you want). It behaves exactly like the old machine and takes about half the time:

Part	Needed	Not needed (identical copy only)
1. Hardware	CPU, IP address, **system memory byte** at address 1	Clock memory byte
2. Tags	5 inputs (I0.1 - I0.5), 6 outputs (Q0.0 - Q0.5), 7 flags (M2.0 - M2.6). One tag table is enough	The I0.0 sensor tag, 8 spare input tags, 4 spare output tags, separate DI / DO tables
3. Data blocks	DB_HMI: 3 buttons. DB_ALARMS: all 18 rows. DB_AUTO: ACTIVE, CONVEYOR ON, AUTO STATUS. DB_CAMERA: DATA1 - DATA10. DB_TCP_IP: MY CONNECT DATA	DB_HMI: STOP TIME, CAMERA DONE. DB_AUTO: 6 old-cycle rows. DB_CAMERA: DATA11 - DATA19. DB_TCP_IP: 3 counters
4 - 7. FC_GENERAL, FC_MANUAL, FC_ALARMS, FC_OUTPUTS	Every network	Network titles, the empty last network
8. FC_AUTO	**Way A: 3 short networks**	Way B: the 6 original networks with the switched-off sensor cycle
8b. FC_TCP_IP	Network 1 (connection settings), network 2 (MB_SERVER)	Network 3 (message counters)
9. OB1	The six block calls, in order	The empty network 7
10. HMI	Connection, FAULT / RESET button, AUTO and MANUAL buttons, the two JOG buttons. Useful: mode field, status field, alarms	Date / time, title, coloured bars, the two I/O status screens with their 24 lamps, exact colours and positions
12. Red-button reverse	-	Optional new feature

If you choose the minimum build, keep it consistent: an item marked "old sensor cycle only" is needed by FC_AUTO way B and by nothing else. Way A + no old-cycle rows, or way B + all old-cycle rows.

Ladder symbols used in this guide



- **Normally open contact:** lets the signal pass when the tag is 1 (ON).
- **Normally closed contact:** lets the signal pass when the tag is 0 (OFF).
- **Coil:** the tag is 1 while signal arrives, 0 when it does not.
- **Set coil (S):** turns the tag ON and it stays ON. **Reset coil (R):** turns it OFF.
- Contacts in a line mean AND. Contacts one below the other (a branch) mean OR.

In TIA Portal the symbols are in the toolbar above the network and in the **Instructions** panel on the right, under **Basic instructions > Bit logic operations**. Drag a symbol onto the line. Then double-click the red **<??.?>** above it and type the tag name. To make a branch: select the line, click **Open branch** (arrow going down) in the toolbar, place the contact, then drag the end arrow of the branch up to the main line to close it.

Part 1. New project and PLC hardware

1 Start TIA Portal V21. Click **Create new project**. Project name: **Conveyor_New**. Click **Create**.

2 Bottom left, click **Project view**.

3 In the project tree (left) double-click **Add new device**.

4 Click **Controllers**. Open **SIMATIC S7-1200 > CPU > CPU 1214C DC/DC/DC > 6ES7 214-1AG40-0XB0**.

5 On the right, **Version**: choose **V4.4** (the old project uses V4.4. If your new CPU has newer firmware, V4.4 still works). Device name stays **PLC_1**. Click **OK**.

6 If a security wizard opens, keep the default choices or untick the protection options and finish. The old project has no PLC password.

7 The device view opens. Double-click the picture of the CPU. The **Properties** window opens at the bottom, tab **General**.

8 Click **PROFINET interface [X1] > Ethernet addresses**. Click **Add new subnet** (it becomes PN/IE_1). Tick **Set IP address in the project**. IP address **192.168.0.40**, subnet mask **255.255.255.0**.

9 Click **System and clock memory**. Tick **Enable the use of system memory byte**, address **1**: this is **needed**. **Enable the use of clock memory byte**, address **0**: **not needed**, the program never uses it; tick it only for an identical copy.

10 Click **DI 14/DQ 10 > I/O addresses**. Check: input start address 0, output start address 0.

11 Press **Ctrl + S** to save.

Why the system and clock memory bytes? They create ready-made tags. The system memory byte at 1 gives **AlwaysTRUE = M1.2** and **AlwaysFALSE = M1.3**; the program uses both. The clock memory byte at 0 gives eight blinking bits, from Clock_10Hz (M0.0) to Clock_0.5Hz (M0.7); part 2 lists them all. The old program does not use them, but they are switched on in the old project, so copy that. With other addresses the M2.x flags could clash.

Part 2. PLC tags

The old project keeps its tags in **three tables**: **DI** (inputs), **DO** (outputs) and the **Default tag table** (system and clock tags, plus the 7 internal flags). The column **Needed?** tells you what the machine really uses. **Minimum:** type only the YES rows; you may put them all into the Default tag table (three tables are only tidier). **Identical copy:** type every row.

1. Project tree: **PLC_1 > PLC tags**. For the identical copy: double-click **Add new tag table**, right-click the new table > **Rename** > type **DI**; do the same again for **DO**. For the minimum: skip this step and use the Default tag table.
2. Open the table. Click the first empty row (<Add new>), type the name, choose data type **Bool**, type the address.
3. The Default tag table already holds the system tags made by part 1. Leave them, and add the 7 flags.

Inputs (tag table DI)

Name	Data type	Address	Meaning	Needed?
DI_PRODUCT SENSOR	Bool	%I0.0	Old sensor input, nothing wired to it	NO - old sensor cycle only
DI_AUTO MANUAL SELECTION	Bool	%I0.1	1 = AUTO, 0 = MANUAL	YES
DI_CYCLE START PB	Bool	%I0.2	Green start button (NO contact)	YES
DI_CYCLE STOP PB	Bool	%I0.3	Red stop button (NC contact, so 1 = not pressed)	YES
DI_EMERGENCY PB	Bool	%I0.4	Emergency (NC contact, so 1 = healthy)	YES
DI_CONVEYOR VFD FAULT	Bool	%I0.5	1 = drive has a fault	YES
DI_SPARE I0.6	Bool	%I0.6	Spare, used nowhere	NO - only for an identical copy
DI_SPARE I0.7	Bool	%I0.7	Spare, used nowhere	NO - only for an identical copy
DI_SPARE I1.0	Bool	%I1.0	Spare, used nowhere	NO - only for an identical copy
DI_SPARE I1.1	Bool	%I1.1	Spare, used nowhere	NO - only for an identical copy
DI_SPARE I1.2	Bool	%I1.2	Spare, used nowhere	NO - only for an identical copy
DI_SPARE I1.3	Bool	%I1.3	Spare, used nowhere	NO - only for an identical copy
DI_SPARE I1.4	Bool	%I1.4	Spare, used nowhere	NO - only for an identical copy
DI_SPARE I1.5	Bool	%I1.5	Spare, used nowhere	NO - only for an identical copy

Outputs (tag table DO)

Name	Data type	Address	Meaning	Needed?
DO_CONVEYOR FWD ON	Bool	%Q0.0	Drive: run forward	YES
DO_CONVEYOR REV ON	Bool	%Q0.1	Drive: run reverse	YES
DO_FAULT RESET	Bool	%Q0.2	Drive: reset fault	YES
DO_TL GREEN	Bool	%Q0.3	Tower lamp green	YES
DO_TL RED	Bool	%Q0.4	Tower lamp red	YES
DO_TL AMBER	Bool	%Q0.5	Tower lamp amber	YES
DO_SPARE Q0.6	Bool	%Q0.6	Spare, used nowhere	NO - only for an identical copy
DO_SPARE Q0.7	Bool	%Q0.7	Spare, used nowhere	NO - only for an identical copy
DO_SPARE Q1.0	Bool	%Q1.0	Spare, used nowhere	NO - only for an identical copy
DO_SPARE Q1.1	Bool	%Q1.1	Spare, used nowhere	NO - only for an identical copy

Type each name exactly, with the space: for example **DI_SPARE I0.6** (DI_SPARE, one space, I0.6).

Default tag table: rows that TIA makes by itself (do not type them)

Name	Data type	Address	Made by	Needed?
System_Byte	Byte	%MB1	System memory byte	YES
FirstScan	Bool	%M1.0	System memory byte	NO - not used, but comes with the byte
DiagStatusUpdate	Bool	%M1.1	System memory byte	NO - not used, but comes with the byte
AlwaysTRUE	Bool	%M1.2	System memory byte	YES - used by the program
AlwaysFALSE	Bool	%M1.3	System memory byte	YES - used by the program
Clock_Byte	Byte	%MB0	Clock memory byte	NO - only for an identical copy
Clock_10Hz	Bool	%M0.0	Clock memory byte	NO - only for an identical copy
Clock_5Hz	Bool	%M0.1	Clock memory byte	NO - only for an identical copy
Clock_2.5Hz	Bool	%M0.2	Clock memory byte	NO - only for an identical copy
Clock_2Hz	Bool	%M0.3	Clock memory byte	NO - only for an identical copy
Clock_1.25Hz	Bool	%M0.4	Clock memory byte	NO - only for an identical copy
Clock_1Hz	Bool	%M0.5	Clock memory byte	NO - only for an identical copy
Clock_0.625Hz	Bool	%M0.6	Clock memory byte	NO - only for an identical copy
Clock_0.5Hz	Bool	%M0.7	Clock memory byte	NO - only for an identical copy

The first five rows appear when you tick the **system memory byte** (address 1) in part 1: this is needed. The clock rows appear only if you also tick the **clock memory byte** (address 0): the program does not use them.

Default tag table: add these 7 flags (all needed)

Name	Data type	Address	Meaning	Needed?
CONTROL ON FLAG	Bool	%M2.0	Emergency is healthy	YES
MANUAL MODE SELECTED	Bool	%M2.1		YES
AUTO MODE SELECTED	Bool	%M2.2	Also shown on the HMI	YES
FAULT RESET FLAG	Bool	%M2.3	Reset button on the HMI is pressed	YES
FAULT BIT	Bool	%M2.4	Any alarm is present	YES
MANUAL CONVEYOR FWD ON FLAG	Bool	%M2.5		YES
MANUAL CONVEYOR REV ON FLAG	Bool	%M2.6		YES

Part 3. Data blocks (DB)

A data block is a table of values kept in the PLC. This project has five that you create by hand and one that TIA creates for you (in part 8b). The DB **numbers matter** because the HMI and Modbus use them.

How to create each DB

1. Project tree: **PLC_1 > Program blocks > Add new block**.
2. Click **Data block (DB)**. Type the **Name**. Type: **Global DB**. Number: choose **Manual** and type the number. Click **OK**.
3. Before adding rows: in the project tree right-click the new DB > **Properties > Attributes**. For **DB_HMI, DB_ALARMS, DB_AUTO and DB_CAMERA**: **untick "Optimized block access"**, click OK and confirm the message. For **DB_TCP_IP**: leave it **ticked** (optimized), as in the old project.
4. In the DB editor add the rows from the table. Type the name, then the data type. Leave the Start value as it is.
5. Click **Compile** (toolbar, or right-click the DB > Compile > Software). In the four non-optimized DBs the **Offset** column fills in. Compare it with the offsets in the tables below. (An optimized DB has no offsets.)

Do not skip step 3. With "Optimized block access" ticked the values have no fixed address. Then Modbus cannot use DB_CAMERA, the alarm word DB2.DBW0 does not exist, and the HMI addresses below do not work. The old project has it unticked on these four DBs. Only DB_TCP_IP is optimized: it is used only by name, so it needs no fixed addresses.

The column **Needed?** shows what the machine uses. Rows marked "old sensor cycle only" are used only by the switched-off networks of FC_AUTO (part 8): leave them out in the minimum build, type them for the identical copy.

DB_HMI - number 1

Name	Data type	Offset	Used for	Needed?
FAULT RESET PB	Bool	0.0	HMI fault reset button	YES
MOTOR JOG FWD PB	Bool	0.1	HMI jog forward button	YES
MOTOR JOG REV PB	Bool	0.2	HMI jog reverse button	YES
STOP TIME	DInt	2.0	Delay in seconds of the old sensor cycle	NO - old sensor cycle only
CAMERA DONE	Bool	6.0	Old sensor cycle	NO - old sensor cycle only

DB_ALARMS - number 2 (type all rows)

Name	Data type	Offset	Used for	Needed?
EMERGENCY PB PRESSED FAULT	Bool	0.0	Alarm 1	YES
VFD FAULT	Bool	0.1	Alarm 2	YES
SPARE1	Bool	0.2	Free alarm bit	YES - fills the 16-bit alarm word
SPARE2	Bool	0.3	Free alarm bit	YES - fills the 16-bit alarm word
SPARE3	Bool	0.4	Free alarm bit	YES - fills the 16-bit alarm word
SPARE4	Bool	0.5	Free alarm bit	YES - fills the 16-bit alarm word
SPARE5	Bool	0.6	Free alarm bit	YES - fills the 16-bit alarm word
SPARE6	Bool	0.7	Free alarm bit	YES - fills the 16-bit alarm word
SPARE7	Bool	1.0	Free alarm bit	YES - fills the 16-bit alarm word
SPARE8	Bool	1.1	Free alarm bit	YES - fills the 16-bit alarm word
SPARE9	Bool	1.2	Free alarm bit	YES - fills the 16-bit alarm word
SPARE10	Bool	1.3	Free alarm bit	YES - fills the 16-bit alarm word
SPARE11	Bool	1.4	Free alarm bit	YES - fills the 16-bit alarm word
SPARE12	Bool	1.5	Free alarm bit	YES - fills the 16-bit alarm word
SPARE13	Bool	1.6	Free alarm bit	YES - fills the 16-bit alarm word
SPARE14	Bool	1.7	Free alarm bit	YES - fills the 16-bit alarm word
FAULT POP	Bool	2.0	Tells the HMI that a fault exists	YES
WARNING POP	Bool	2.1	Never set by the program	NO - only for an identical copy

The 14 spare rows are needed even though no alarm uses them: they fill bytes 0 and 1, so that the alarms form one 16-bit word (DBW0) and FAULT POP lands at 2.0.

DB_AUTO - number 3

Name	Data type	Offset	Used for	Needed?
ACTIVE	Bool	0.0	Auto cycle is started	YES
CONVEYOR ON	Bool	0.1	Auto wants the conveyor to run	YES
CONVEYOR OFF1	Bool	0.2	Old sensor cycle	NO - old sensor cycle only
CONVEYOR OFF2	Bool	0.3	Old sensor cycle	NO - old sensor cycle only
P_TRG	Bool	0.4	Old sensor cycle	NO - old sensor cycle only
CAMERA DONE	Bool	0.5	Old sensor cycle	NO - old sensor cycle only
AUTO STATUS	Int	2.0	10 / 20 / 30 for the HMI text	YES
STOP DELAY TIMER	IEC_TIMER	4.0	Old sensor cycle (type the data type by hand)	NO - old sensor cycle only
SENSOR CHECK DELAY	IEC_TIMER	20.0	Old sensor cycle	NO - old sensor cycle only

In the minimum build (only ACTIVE, CONVEYOR ON, AUTO STATUS) the offsets are 0.0, 0.1 and 2.0: the same as in the full table.

DB_CAMERA - number 5

All data type **Int**. Tip: make DATA1, select the row, and drag the small square at the bottom-left corner of the row down; TIA numbers the copies for you. Then check every row against this table.

Name	Data type	Offset	Modbus register	Needed?
DATA1	Int	0.0	40001	YES - the PC writes 0 = run, 1 = stop
DATA2	Int	2.0	40002	YES - the Modbus block needs 10 values
DATA3	Int	4.0	40003	YES - the Modbus block needs 10 values
DATA4	Int	6.0	40004	YES - the Modbus block needs 10 values
DATA5	Int	8.0	40005	YES - the Modbus block needs 10 values
DATA6	Int	10.0	40006	YES - the Modbus block needs 10 values
DATA7	Int	12.0	40007	YES - the Modbus block needs 10 values
DATA8	Int	14.0	40008	YES - the Modbus block needs 10 values
DATA9	Int	16.0	40009	YES - the Modbus block needs 10 values
DATA10	Int	18.0	40010	YES - the Modbus block needs 10 values
DATA11	Int	20.0	-	NO - only for an identical copy
DATA12	Int	22.0	-	NO - only for an identical copy
DATA13	Int	24.0	-	NO - only for an identical copy
DATA14	Int	26.0	-	NO - only for an identical copy
DATA15	Int	28.0	-	NO - only for an identical copy
DATA16	Int	30.0	-	NO - only for an identical copy
DATA17	Int	32.0	-	NO - only for an identical copy
DATA18	Int	34.0	-	NO - only for an identical copy
DATA19	Int	36.0	-	NO - only for an identical copy

DB_TCP_IP - number 12 (optimized: leave "Optimized block access" ticked)

Name	Data type	Used for	Needed?
MY CONNECT DATA	TCON_IP_v4	Connection settings for Modbus. Type TCON_IP_v4 in the data type column by hand; it then opens into sub-rows.	YES
NO OF DONE	UDInt	Counter of good Modbus messages	NO - fault finding only
NO OF ERROR	UDInt	Counter of failed messages	NO - fault finding only
ERROR STATUS	UDInt	Last error code	NO - fault finding only

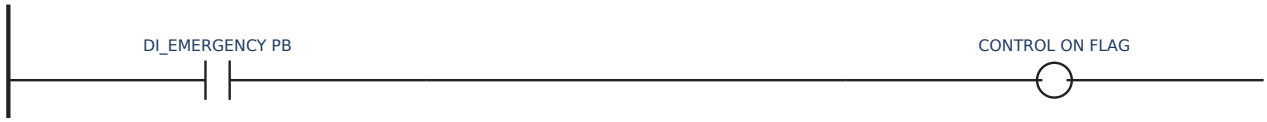
Part 4. Block FC_GENERAL (FC1) - mode and reset

How to create an FC (same for all six)

1. **Program blocks > Add new block**. Click **Function (FC)**.
2. Name as given. Language: **LAD**. Number: **Manual**, type the number. Click **OK**.
3. The editor opens with Network 1. To add a network: right-click a network title > **Insert network**.
4. **Not needed, only for an identical look:** give every network the title shown in this guide (click the grey title line of the network, where it says "Network 1:", and type it).
5. **Not needed:** the old blocks each end with one empty network. It does nothing.

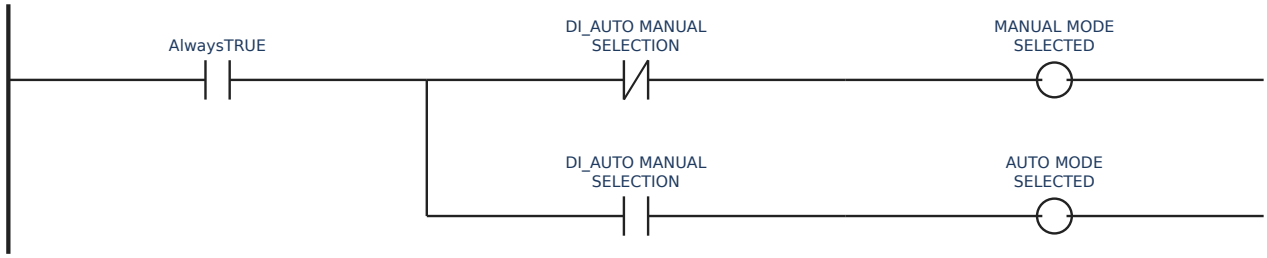
Every drawing in parts 4 - 8b was read from the old project through TIA Portal itself (export of 12 October 2026): same networks, same order, same contacts and branches as the original.

FC_GENERAL - Network 1: CONTROL ON



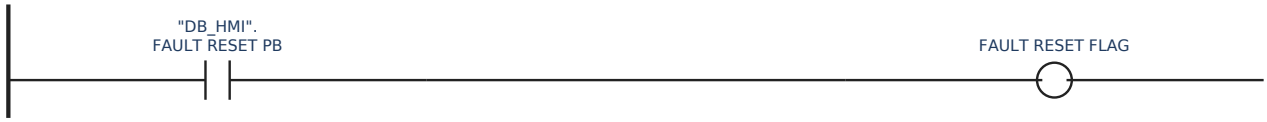
Emergency healthy (input ON) gives CONTROL ON. Pushing Emergency drops it.

FC_GENERAL - Network 2: MODE SELECTION



Selector OFF = manual, ON = auto. The AlwaysTRUE contact is only there so that two coils can share one network.

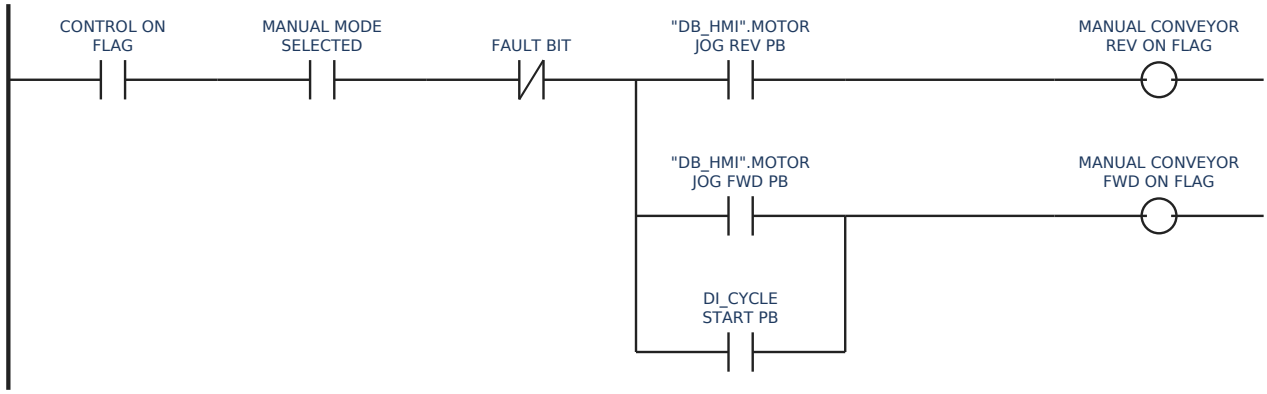
FC_GENERAL - Network 3: FAULT RESET



To use a DB value, type **"DB_HMI".** and pick the row from the list that appears. Network 4 is empty.

Part 5. Block FC_MANUAL (FC2) - jog

FC_MANUAL - Network 1: MOTOR FWD /REV ON

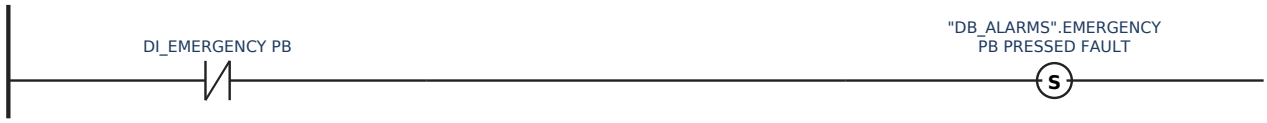


Exactly as in the old block: after the three common contacts the network splits. The **top** row is the HMI reverse button to the REV flag. Below it, the HMI forward button **or** the green START button gives the FWD flag. The motor runs only while a button is held. Network 2 is empty.

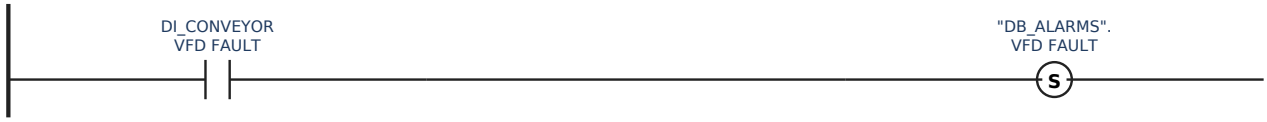
> **Red button reverse:** the old file has **no** red-button reverse (confirmed from the TIA export). You told me it was wanted but never built. Build this block exactly as above first. Part 12 shows how to add the red-button reverse jog safely, as a separate step.

Part 6. Block FC_ALARMS (FC3) - alarms

FC_ALARMS - Network 1: EMERGENCY PB PRESSED FAULT (I0.4)

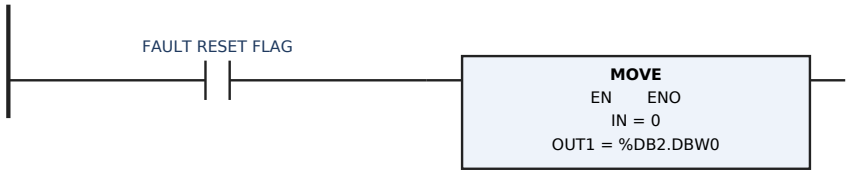


FC_ALARMS - Network 2: CONVEYOR VFD FAULT (I0.5)



Set coils: the alarm stays ON even after the cause has gone, until someone presses reset.

FC_ALARMS - Network 3: FAULT RESET



MOVE is in **Basic instructions > Move operations**. At IN type **0**. At OUT1 type **%DB2.DBW0**. DBW0 is the first 16 bits of DB_ALARMS = all 16 alarm bits together. Writing 0 clears them all at once.

FC_ALARMS - Network 4: FAULT BIT



Compare is in **Basic instructions > Comparator operations > CMP <>**. Click the **???** inside the box and choose **Word**. If any alarm bit is 1 the word is not 0, so FAULT BIT turns ON. The compare box sits directly on the left rail. Network 5 is empty.

Part 7. Block FC_OUTPUTS (FC4) - outputs

FC_OUTPUTS - Network 1: CONVEYOR FWD ON



Networks 2 to 6 are one contact and one coil each. Network 7 is empty.

Network	Title	Contact (normally open)	Coil
2	CONVEYOR REV ON	MANUAL CONVEYOR REV ON FLAG	DO_CONVEYOR REV ON
3	FAULT RESET	FAULT RESET FLAG	DO_FAULT RESET
4	TL GREEN	"DB_AUTO".ACTIVE	DO_TL GREEN
5	TL RED	FAULT BIT	DO_TL RED
6	TL AMBER	MANUAL MODE SELECTED	DO_TL AMBER

Good habit: each physical output (Q) is written in one place only, here. Other blocks only set flags. This avoids two rungs fighting over one output.

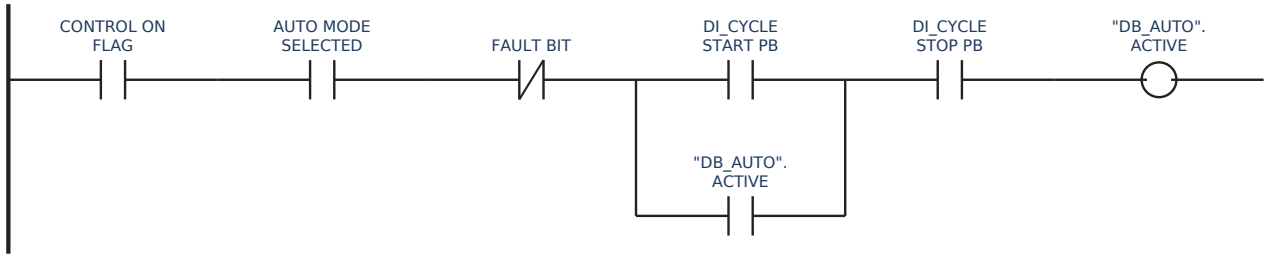
Part 8. Block FC_AUTO (FC5) - automatic cycle

The old block has **six networks**. Only three of them do any work on your machine; the others are an old sensor cycle that is **switched off** by a normally closed AlwaysTRUE contact (it can never pass). So there are two ways to build this block, and **both behave exactly the same**:

Way	What you build	Time
A. Minimum (recommended if you only want a working machine)	The 3 short networks drawn just below	15 minutes
B. Identical copy of the old block	All 6 networks, drawn after that, with the switched-off rows	1 hour

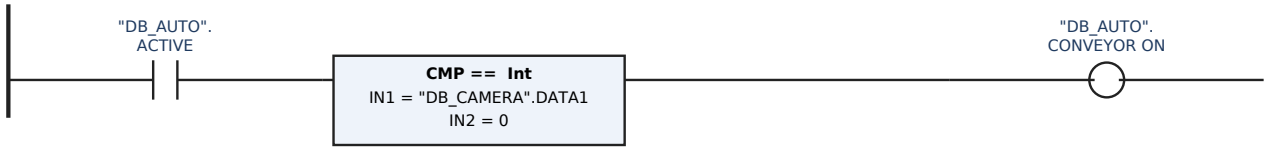
Way A: the minimum FC_AUTO (3 networks, all needed)

FC_AUTO short - Network 1: AUTO ON [NEEDED]



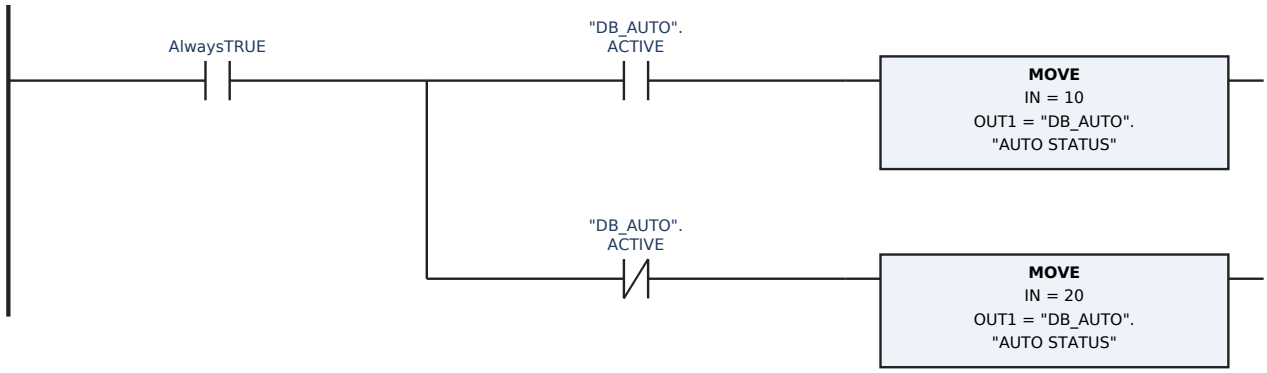
Press START: ACTIVE turns ON and holds itself (a "latch"). It drops on STOP, a fault, Emergency, or when the selector leaves AUTO.

FC_AUTO short - Network 2: CONVEYOR ON [NEEDED]



The belt runs while the cycle is started and register 40001 is 0. This is row 1 of the old network 3 without its two contacts that change nothing.

FC_AUTO short - Network 3: AUTO STATUS [NEEDED for the HMI status text]



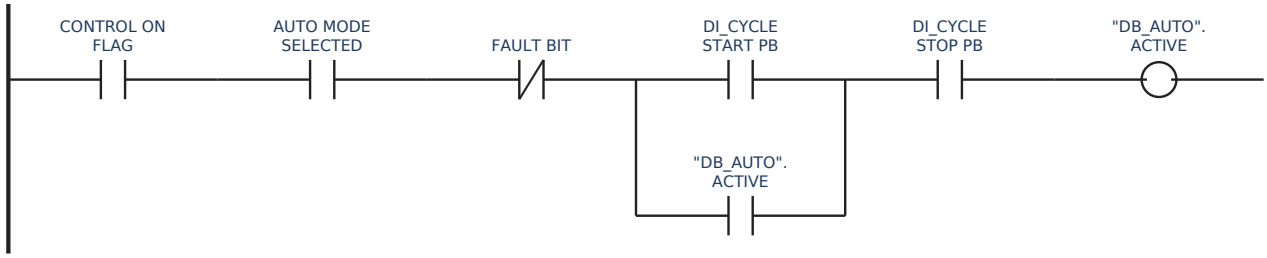
10 = AUTO RUNNING, 20 = STOPPED. With way A you can leave out all "old sensor cycle only" rows of part 2 and part 3, and the SCL network.

If you built way A, **skip the rest of part 8** and go on to part 8b.

Way B: the identical copy (6 networks)

Network 3 is large: its first row is the rung that really runs the belt; the other rows are the switched-off cycle. Each title below says whether the machine needs it.

FC_AUTO - Network 1: AUTO ON [NEEDED]



Press START: ACTIVE turns ON and holds itself through its own contact (a "latch"). It drops when STOP is pressed (the input goes OFF because the button is NC), on a fault, on Emergency, or when the selector leaves AUTO.

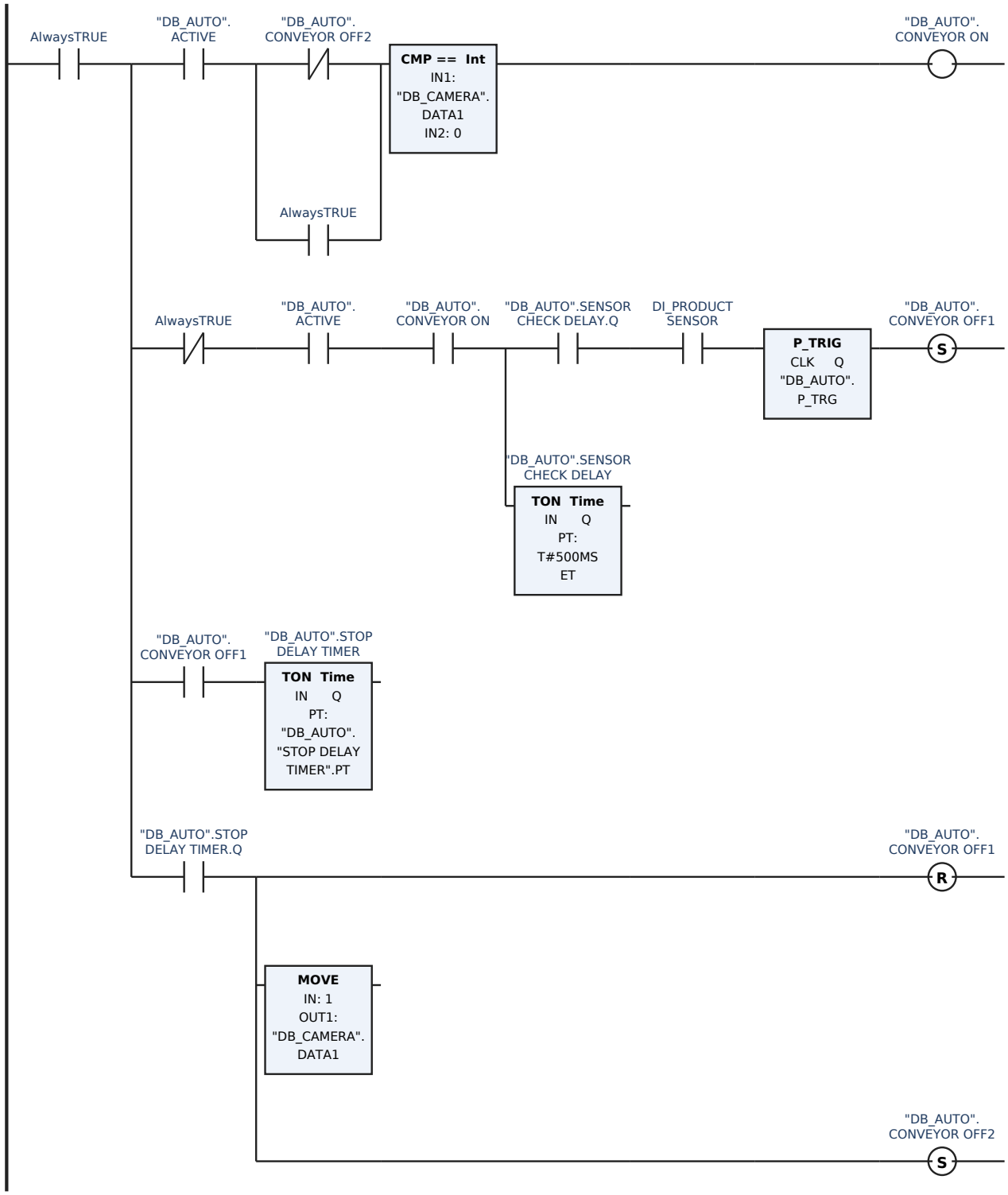
FC_AUTO - Network 2: STOP DELAY TIME (an SCL network) [NOT needed: old sensor cycle]

Right-click the title of network 1 > **Insert SCL network** (this network is text, not ladder). Type this one line:

```
"DB_AUTO"."STOP DELAY TIMER".PT := "DB_HMI"."STOP TIME" * 1000;
```

It turns STOP TIME (seconds, from the HMI) into the timer preset in milliseconds. It belongs to the switched-off sensor cycle.

FC_AUTO - Network 3: CONVEYOR ON [row 1 NEEDED; rows 2 - 8 NOT needed: old sensor cycle]

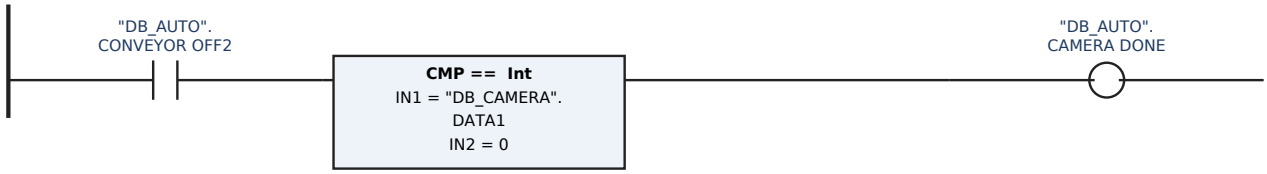


Row 1 runs the belt: AlwaysTRUE, ACTIVE, then (NOT OFF2 **or** AlwaysTRUE), then DATA1 = 0. The AlwaysTRUE branch makes the OFF2 contact have no effect, so: **belt runs while ACTIVE and register 40001 is 0.** The other rows hang from the first AlwaysTRUE contact. The row with the **normally closed AlwaysTRUE** never passes, so the sensor timer, P_TRIG and OFF1 never switch; therefore the stop-delay timer, the MOVE 1 and OFF2 never switch either.

How to place the boxes: CMP == is in **Comparator operations** (choose **Int**). TON is in **Timer operations**: when "Call options" opens, click **Cancel**, then type the instance above the box (for example **"DB_AUTO"."SENSOR CHECK DELAY"**). P_TRIG is in **Bit logic operations**; type **"DB_AUTO".P_TRG** under it. Each new row is a branch: select the wire after AlwaysTRUE (or after the STOP DELAY TIMER.Q contact), click **Open branch**, place the elements, and leave the row

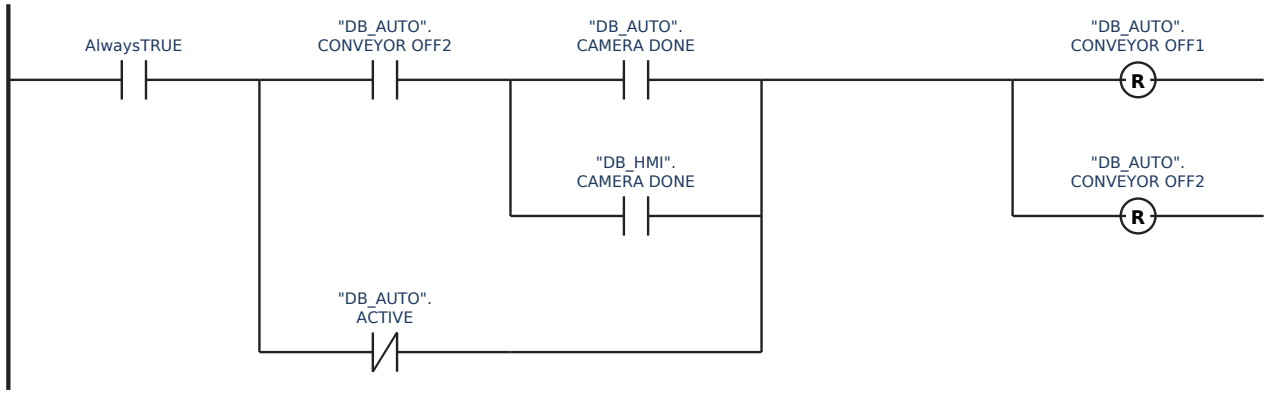
ending in its own coil or box.

FC_AUTO - Network 4: CAMERA DONE [NOT needed: old sensor cycle]



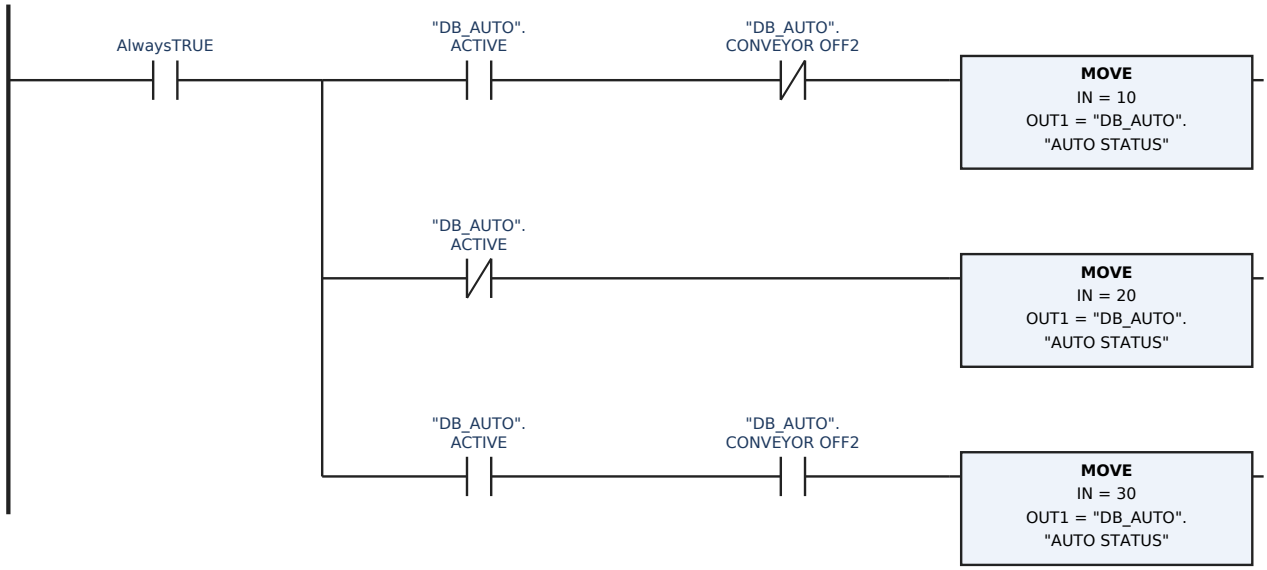
Part of the switched-off cycle (OFF2 is never ON).

FC_AUTO - Network 5: CONVEYOR OFF RESET [NOT needed: old sensor cycle]



Resets OFF1 and OFF2 when the camera step is done (from the PLC or from the HMI) or when the cycle is not active. Part of the switched-off cycle.

FC_AUTO - Network 6: AUTO STATUS [rows 10 and 20 NEEDED; row 30 NOT needed]



10 = started (HMI text AUTO RUNNING), 20 = not started (STOPPED), 30 = waiting for camera (never happens now, because OFF2 stays OFF). Network 7 is empty.

Part 8b. Block FC_TCP_IP (FC6) - Modbus TCP server

Network 1: Connection definition [NEEDED]

Nine MOVE boxes and one contact with a coil (10 rows), **each row starting directly at the left rail** (use **Open branch** for each new row), in this order. For OUT1 type **"DB_TCP_IP"."MY CONNECT DATA".** and pick the sub-row.

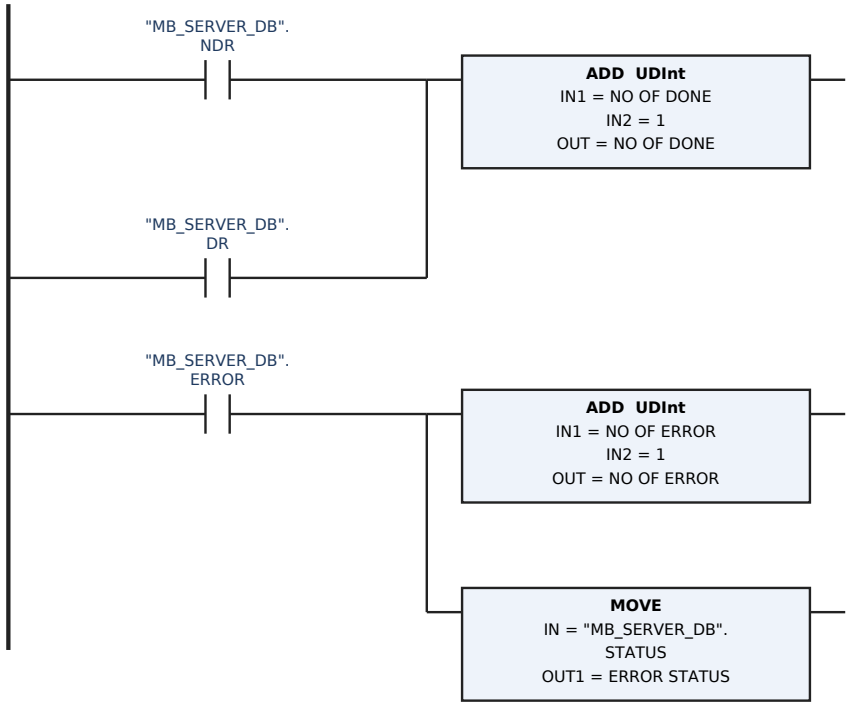
Row	Element	IN	OUT1 / coil = "DB_TCP_IP"."MY CONNECT DATA". ...	Meaning
1	MOVE	"Local~PROFINET_interface_1"	InterfaceId	Which Ethernet port. Pick this name from the list; it is created with the CPU (value 64).
2	MOVE	1	ID	Connection number
3	MOVE	11	ConnectionType	11 = TCP
4	Normally closed contact **AlwaysTRUE**, then coil	-	ActiveEstablished	Never passes, so 0 = the PLC waits (server)
5	MOVE	0	RemoteAddress.ADDR[1]	Remote IP 0.0.0.0 = accept any device
6	MOVE	0	RemoteAddress.ADDR[2]	
7	MOVE	0	RemoteAddress.ADDR[3]	
8	MOVE	0	RemoteAddress.ADDR[4]	
9	MOVE	0	RemotePort	Any port on the other side
10	MOVE	502	LocalPort	The Modbus port of the PLC

Network 2: MB SERVER [NEEDED]

1. Instructions panel: **Communication > Others > MODBUS TCP**. In the **Version** column of the MODBUS TCP line choose **V5.3** (the old project uses MB_SERVER V5.3; TIA may offer a newer version by default).
2. Drag **MB_SERVER** onto the line. "Call options" opens. Choose **Single instance**. Name **MB_SERVER_DB**. Number **Manual**, **4**. Click **OK**. TIA creates DB4 by itself.
3. Fill the inputs on the left of the box as in the table. Leave all outputs on the right empty.

Pin	Type this	Meaning
DISCONNECT	AlwaysFALSE	0 = keep listening for a connection
MB_HOLD_REG	P#DB5.DBX0.0 INT 10	Registers 40001 - 40010 are the first 10 values of DB_CAMERA (DB5)
CONNECT	"DB_TCP_IP"."MY CONNECT DATA"	The settings from network 1

FC_TCP_IP - Network 3: ERROR CHECK [NOT needed: counters for fault finding only]



ADD is in **Math functions**; choose UDInt inside the box. NO OF DONE, NO OF ERROR and ERROR STATUS are in "DB_TCP_IP". After the ERROR contact the line splits: ADD and MOVE sit on two parallel branches. These counters are only for fault finding: watch them go up when the PC talks to the PLC.

Part 9. Main block OB1 and compile

1. Open **Program blocks > Main [OB1]**.
2. Drag **FC_AUTO** from the project tree onto the line of Network 1. A box "FC_AUTO" appears.
3. Insert five more networks and drag one block into each, in this order: **FC_ALARMS, FC_GENERAL, FC_MANUAL, FC_OUTPUTS, FC_TCP_IP**. (The old OB1 also has an empty network 7. Not needed.)
4. In the project tree click **PLC_1**. Click **Compile** in the toolbar.
5. Look at the **Info > Compile** tab at the bottom. It must end with **Errors: 0**. Double-click any red line to jump to the mistake. Usual causes: a name typed differently, a contact left with <??.?>, or one of the four DBs still "optimized".
6. Save with **Ctrl + S**.

Check the block list. Program blocks must now show: Main [OB1], FC_GENERAL [FC1], FC_MANUAL [FC2], FC_ALARMS [FC3], FC_OUTPUTS [FC4], FC_AUTO [FC5], FC_TCP_IP [FC6], DB_HMI [DB1], DB_ALARMS [DB2], DB_AUTO [DB3], MB_SERVER_DB [DB4], DB_CAMERA [DB5], DB_TCP_IP [DB12].

Part 10. HMI panel (KTP700 Basic)

Everything in this part was exported from the old project by TIA Portal itself (12 October 2026): screen names and numbers, positions, sizes, texts, colours, tags, text lists, button functions and animations. The panel has **five screens** and one **template called MAIN**. The template holds the title bar and the row of five buttons at the bottom. The screen is 800 x 480 points. Positions are **x, y** of the top-left corner and **width x height**; type them under **Properties > Layout**. Colours are given as red, green, blue numbers (Properties > Appearance).

Two things the export cannot show: alarm views and the discrete alarm list (TIA's export does not include them). They are given in 10.3 and 10.4 from older screen files of the same project (6 February 2026), marked **CHECK**. Look at the ALARMS screen of the old panel to confirm.

10.1 Add the panel and connect it to the PLC

1. Project tree: **Add new device > HMI > SIMATIC Basic Panel > 7" Display > KTP700 Basic > 6AV2 123-2GB03-0AX0**. Device name **HMI_1**. Leave "Start device wizard" ticked. Click OK. (Its runtime is called HMI_RT_1, the TIA default.)
2. Wizard page "PLC connections": click **Browse**, choose **PLC_1**. On the next pages untick the ready-made screens and system screens, and click **Finish**.
3. Project tree: **Devices & networks**. Double-click the HMI. Properties: **PROFINET Interface [X1] > Ethernet addresses**: subnet PN/IE_1, IP **192.168.0.41**, mask 255.255.255.0.
4. Check **HMI_1 > Connections**: one row **HMI_Connection_1** to PLC_1 must exist.

10.2 HMI tags: three tag tables

Open **HMI_1 > HMI tags**. Double-click **Add new tag table** twice and rename the new tables **IO** and **ALARMS**. Connection is always **HMI_Connection_1**, acquisition cycle **1 s**. For rows with a PLC name, click the "..." button in the **PLC tag** column and pick it. For rows with an address, choose Access mode **<absolute access>** and type the address.

Table	HMI tag name	Data type	PLC tag or address	Used for	Needed?
Default	Tag_ScreenNumber	UInt	(internal, made by the wizard; no connection)	Leave it	NO - made by the wizard
Default	AUTO MODE SELECTED	Bool	PLC tag AUTO MODE SELECTED (%M2.2)	Mode text AUTO / MANUAL	USEFUL - mode text
Default	DB_ALARMS_FAULT POP	Bool	"DB_ALARMS"."FAULT POP"	Red alarm line	USEFUL - alarm line
Default	DB_ALARMS_WARNIN G POP	Bool	"DB_ALARMS"."WARNING POP"	Warning line (the PLC never sets it)	NO - never set
Default	DB_HMI_FAULT RESET PB	Bool	"DB_HMI"."FAULT RESET PB"	FAULT / RESET button	YES
Default	DB_HMI_MOTOR JOG FWD PB	Bool	"DB_HMI"."MOTOR JOG FWD PB"	FWD JOG button	YES
Default	DB_HMI_MOTOR JOG REV PB	Bool	"DB_HMI"."MOTOR JOG REV PB"	REV JOG button	YES
Default	DB_AUTO_AUTO STATUS	Int	"DB_AUTO"."AUTO STATUS"	Status text on the AUTO screen	USEFUL - status text
Default	DB_HMI_STOP TIME	DInt	"DB_HMI"."STOP TIME"	Old sensor cycle; on no screen	NO - on no screen
Default	DB_HMI_CAMERA DONE	Bool	"DB_HMI"."CAMERA DONE"	Old sensor cycle; on no screen	NO - on no screen

Table	HMI tag name	Data type	PLC tag or address	Used for	Needed?
IO	IB0	Byte	%IB0 (absolute)	Lamps I0.0 - I0.7	NO - only for the I/O screens
IO	IB1	Byte	%IB1 (absolute)	Lamps I1.0 - I1.5	NO - only for the I/O screens
IO	QB0	Byte	%QB0 (absolute)	Lamps Q0.0 - Q0.7	NO - only for the I/O screens
IO	QB1	Byte	%QB1 (absolute)	Lamps Q1.0 - Q1.1	NO - only for the I/O screens
ALARMS	ALARM WORD	**Int**	%DB2.DBW0 (absolute). Acquisition mode: **Cyclic continuous**	Alarm messages	USEFUL - alarm texts

All tags except ALARM WORD use acquisition mode **Cyclic in operation** (the default).

10.3 Text lists and alarms

Needed? The two text lists are needed by the mode field and the status field (if you build those fields). The discrete alarms are **useful**: without them the red lamp still works, but the panel does not say which alarm it is.

Where	What to enter
HMI_1 > Text and graphic lists > Text lists	List **AUTO STATUS** (with a space), selection "Value/Range": 10 = **AUTO RUNNING**, 20 = **STOPPED**, 30 = **WAITING FOR CAMERA ACK**. List **MODE**, selection "Bit (0, 1)": 0 = **MANUAL**, 1 = **AUTO**.
HMI_1 > HMI alarms > Discrete alarms (CHECK, see the box above)	Row 1: alarm text **EMERGENCY PB PRESSED FAULT (I0.4)**, alarm class Errors, trigger tag **ALARM WORD**, trigger bit **8**. Row 2: **CONVEYOR VFD FAULT (I0.5)**, trigger tag ALARM WORD, trigger bit **9**.

Why bits 8 and 9, not 0 and 1? In a Siemens word the first byte (DBB0) is the high half. So DBX0.0 is bit 8 of DBW0 and DBX0.1 is bit 9. TIA shows the matching address (DB2.DBX0.0) next to the trigger bit; use that to check.

10.4 The template MAIN (shown on every screen)

Open **HMI_1 > Screen management > Templates**. Rename Template_1 to **MAIN** (right-click > Rename). Background colour **182, 182, 182**. Drag the objects from the **Toolbox** on the right. Buttons: on **Properties > Events > Press** click <Add function> and choose the function shown.

Object	x, y	Size	Settings	Needed?
Rectangle (top bar)	1, 1	798 x 58	Background 192, 192, 192	NO - looks only
Date/time field (time only)	1, 1	121 x 58	Mode Output; show time ON, date OFF	NO - looks only
Date/time field (date only)	677, 1	121 x 58	Mode Output; show date ON, time OFF	NO - looks only
Text field **CONVEYOR**	225, 11	327 x 39	Background 112, 112, 112, text white, centred. This is the title.	NO - looks only
Button without text (dark strip behind the mode line)	0, 61	800 x 30	Background 99, 101, 113 with vertical gradient; no event. It is only decoration.	NO - looks only
Text field **MODE :-**	12, 64	65 x 23	Transparent background	USEFUL
Symbolic I/O field	89, 61	117 x 27	Mode **Output**, process tag **AUTO MODE SELECTED**, text list **MODE**, white background	USEFUL - shows the mode

Object	x, y	Size	Settings	Needed?
Rectangle (bottom bar)	1, 406	799 x 68	Background 198, 195, 198	NO - looks only
Button **AUTO**	6, 411	151 x 60	Event **Press**: Screens > **ActivateScreen**, screen AUTO	YES - to open the screen
Button **MANUAL**	166, 411	151 x 60	Press: ActivateScreen, MANUAL	YES - to open the screen
Button **ALARMS**	326, 411	151 x 60	Press: ActivateScreen, ALARMS	USEFUL
Button **IO STATUS**	486, 411	151 x 60	Press: ActivateScreen, INPUT STATUS	NO - only with the I/O screens
Button **FAULT / RESET** (two lines)	646, 411	151 x 60	Background **yellow 255, 255, 0**. Event **Press**: Edit bits > **SetBitWhileKeyPressed**, tag DB_HMI_FAULT RESET PB, bit 0	YES - without it an alarm can never be cleared
CHECK: alarm view(s) for the red / yellow alarm line	124, 4	550 x 53	Not in the export. The February screen files had two alarm views here (Errors red, Warnings yellow), made visible by DB_ALARMS_FAULT POP / WARNING POP. Copy what the old panel shows.	USEFUL - shows the alarm line

All five navigation and reset buttons have a blue border (0, 0, 255, width 3) and grey background 218, 218, 218, except the yellow FAULT / RESET button.

SetBitWhileKeyPressed sets the bit while the finger is on the button and clears it when the finger is lifted. It needs only the **Press** event. (Revision 2 told you to use SetBit on Press and ResetBit on Release. That also works, but the old project uses SetBitWhileKeyPressed.)

10.5 The five screens

Create each with **HMI_1 > Screens > Add new screen**, give it the name and number shown (Properties > General), and check that its template is **MAIN**. In the old project INPUT STATUS and OUTPUT STATUS are in a screen folder called **IOs** (right-click Screens > Add group).

Screen (number)	Object	x, y	Size	Settings	Needed?
AUTO (1), background white	Text field **AUTO OPERATION**	308, 62	181 x 27	Screen title	NO - looks only
	Text field **AUTO STATUS**	304, 117	165 x 34		NO - looks only
	Symbolic I/O field	116, 182	560 x 106	Mode Output. Process tag DB_AUTO_AUTO STATUS. Text list AUTO STATUS. Border 3. Animation **Appearance**, tag DB_AUTO_AUTO STATUS: value 10 = background **green** (0, 255, 0), text black; value 20 = background **red** (255, 0, 0), text white	USEFUL - shows running / stopped
MANUAL (2), background white	Text field **MANUAL OPERATION**	308, 62	210 x 27	Screen title	NO - looks only

Screen (number)	Object	x, y	Size	Settings	Needed?
	Button **CONVEYOR / FWD JOG** (two lines)	166, 194	179 x 82	Font Tahoma 19. Press: **SetBitWhileKeyPressed** DB_HMI_MOTOR JOG FWD PB, bit 0. Animation Appearance, same tag: value 1 = background green (0, 255, 0)	USEFUL - the green START button does the same
	Button **CONVEYOR / REV JOG** (two lines)	429, 190	179 x 82	Same, with DB_HMI_MOTOR JOG REV PB	YES - the only way to reverse
ALARMS (3), background 182, 182, 182	Text field **ALARMS**	372, 62	84 x 27	Screen title	NO - looks only
	CHECK: Alarm view	2, 92	797 x 308	Not in the export; from the February files. Toolbox > Controls > Alarm view	USEFUL - shows which alarm
INPUT STATUS (10), folder IOs, background 181, 182, 181	Text field **INPUT STATUS**	308, 63	147 x 27	Screen title	NO - fault finding only
	Lamp table (10.6)	0, 94		14 rows, listed one by one in 10.6	NO - fault finding only
	Button **NEXT**	644, 341	151 x 60	Press: ActivateScreen, OUTPUT STATUS	NO - fault finding only
OUTPUT STATUS (19), folder IOs, background 181, 182, 181	Text field **OUTPUT STATUS-1**	308, 63	184 x 27	Screen title (the text ends in -1; the screen name does not)	NO - fault finding only
	Lamp table (10.6)	0, 94		10 rows, listed one by one in 10.6	NO - fault finding only
	Button **BACK**	644, 341	151 x 60	Press: ActivateScreen. **In the old project it points to OUTPUT STATUS itself, so BACK does nothing.** Choose INPUT STATUS to make it work (a small fix).	NO - fault finding only

Start screen: set **HMI_1 > Runtime settings > General > Start screen** = AUTO (screen number 1; the export does not include runtime settings).

10.6 The lamp tables on the two status screens [NOT needed to run the machine]

These two screens are not needed. They only show which input and output is ON, which helps when you look for a wiring fault. They are also the slowest part of the HMI: 24 lamps, about one hour. In the minimum build skip all of 10.6, the IO tag table (IB0, IB1, QB0, QB1) and the IO STATUS button. You can add them later at any time.

Each table is built from text fields and circles. There are two blocks side by side: the left block starts at x = 0, the right block at x = 401.

Piece	Left block x / width	Right block x / width	Look
Header row at y = 94, height 26: **ADDRESS**, **STATUS**, **DESCRIPTION**	0 / 84, 85 / 70, 156 / 241	401 / 84, 486 / 70, 557 / 242	Green background (0, 128, 0), white text

Piece	Left block x / width	Right block x / width	Look
Data rows, height 34 (y of each row: tables below)	Same three columns	Same three columns	Dark grey background (51, 51, 51). Address in yellow text, description in white text. The STATUS cell stays empty.
Circle 23 x 23 in the STATUS column of each row	x = 109	x = 510	Position of each circle: tables below. Grey 217, 217, 217 in the editor.

INPUT STATUS: all 14 rows

Address text	Description text	Block	Row y	Circle x, y	Circle tag	Bit
I0.0	PRODUCT SENSOR	Left	120	109, 126	IB0	0
I0.1	AUTO MANUAL SELECTION	Left	155	109, 161	IB0	1
I0.2	CYCLE START PB	Left	191	109, 197	IB0	2
I0.3	CYCLE STOP PB	Left	227	109, 233	IB0	3
I0.4	EMERGENCY PB	Left	263	109, 269	IB0	4
I0.5	CONVEYOR VFD FAULT	Left	299	109, 305	IB0	5
I0.6	SPARE	Left	335	109, 341	IB0	6
I0.7	SPARE	Left	371	109, 377	IB0	7
I1.0	SPARE	Right	120	510, 126	IB1	0
I1.1	SPARE	Right	155	510, 161	IB1	1
I1.2	SPARE	Right	191	510, 197	IB1	2
I1.3	SPARE	Right	227	510, 233	IB1	3
I1.4	SPARE	Right	263	511, 269	IB1	4
I1.5	SPARE	Right	299	511, 304	IB1	5

OUTPUT STATUS: all 10 rows

Address text	Description text	Block	Row y	Circle x, y	Circle tag	Bit
Q0.0	CONVEYOR FWD ON	Left	121	109, 127	QB0	0
Q0.1	CONVEYOR REV ON	Left	156	109, 162	QB0	1
Q0.2	FAULT RESET	Left	192	109, 198	QB0	2
Q0.3	TL GREEN	Left	228	109, 234	QB0	3
Q0.4	TL RED	Left	264	109, 270	QB0	4
Q0.5	TL AMBER	Left	300	109, 306	QB0	5
Q0.6	SPARE	Left	336	109, 342	QB0	6
Q0.7	SPARE	Left	372	109, 378	QB0	7
Q1.0	SPARE	Right	121	510, 127	QB1	0
Q1.1	SPARE	Right	156	510, 162	QB1	1

Each row is three text fields (address, empty status cell, description) at the x positions of its block, and one circle. Example, row I0.3: text **I0.3** at x = 0, y = 227; empty cell at x = 85, y = 227; text **CYCLE STOP PB** at x = 156, y = 227; circle at 109, 233 with tag IB0, bit 3.

Make a circle light up: select the circle > Properties > **Animations > Display > Add new animation > Appearance**. Tag: **IB0** for I0.x (IB1 for I1.x, QB0 for Q0.x, QB1 for Q1.x). Type: **Single bit**, and the bit number = the digit after the dot (I0.3 is bit 3). Then two rows: **0 = red** (255, 0, 0) and **1 = green** (0, 255, 0), background and foreground. So a lamp is red when the signal is OFF and green when it is ON. Build one complete row, then copy and paste it and change only the address text, the description and the bit number.

Last step: click **HMI_1** and **Compile**. Fix errors until it shows Errors: 0.

Part 11. Download and test

11.1 Download

1. Laptop network card: IP 192.168.0.100, mask 255.255.255.0. Connect laptop, PLC and HMI to the switch.
2. Click **PLC_1**. Menu **Online > Extended download to device**. Interface type **PN/IE**, choose your network card, tick **Show all compatible devices**, **Start search**, select the PLC, **Load**. Accept the questions, then **Load**, tick **Start module**, **Finish**.
3. On the panel: Settings > Network Interface: IP 192.168.0.41. Settings > Transfer Settings: Enable transfer ON.
4. Click **HMI_1**. **Online > Extended download to device**, same choices, **Load**. Accept an operating system update if TIA asks.

11.2 Watch your logic working

1. Open a block, for example FC_AUTO. Click the **Monitoring on/off** button (glasses icon) in the block toolbar.
2. Green solid line = signal passes. Blue dotted line = no signal. This is the best way to learn: press a button on the machine and see which contact changes.
3. For values, use **PLC_1 > Watch and force tables > Add new watch table**. Add "DB_CAMERA".DATA1, "DB_AUTO"."AUTO STATUS", "DB_TCP_IP"."NO OF DONE". Click the glasses icon. To change a value type it in **Modify value** and click the lightning icon.

11.3 Test list

#	Do this	You must see
1	Nothing pressed, selector on MANUAL	I0.3 and I0.4 ON. Amber lamp ON. HMI mode text MANUAL.
2	Hold CONVEYOR FWD JOG on the HMI, then CONVEYOR REV JOG	Q0.0 ON while held, then Q0.1 ON while held.
3	Hold the green START button (still MANUAL)	Q0.0 ON while held.
4	Push Emergency	Red lamp ON, alarm text on the HMI. Jog does not work.
5	Release Emergency, press FAULT RESET on the HMI	Alarm clears, red lamp OFF.
6	Selector AUTO, press START	Green lamp ON, Q0.0 ON, HMI status AUTO RUNNING.
7	In the watch table write 1 to "DB_CAMERA".DATA1, then 0 (this is what the PC does)	Q0.0 OFF at 1 (green lamp stays ON). Q0.0 ON again at 0.
8	Press STOP	Green lamp OFF, Q0.0 OFF, HMI status STOPPED.
9	On the PC, inside the folder conveyor_automation-main / conveyor_automation-main, run `python3 backend_testing/test_plc_connection.py`. Type w, then 1. Type w, then 0.	Conveyor stops, then runs. "DB_TCP_IP"."NO OF DONE" counts up.

No hardware yet? You can still practise parts 1 to 10 and compile. If S7-PLCSIM is installed, click PLC_1 and the **Start simulation** button in the toolbar to run the program on the laptop and switch inputs by hand. The Modbus part needs the real PLC.

Common mistakes

What you see	Cause	Fix
Machine never starts in AUTO	I0.3 or I0.4 is OFF	STOP and Emergency must be wired through NC contacts.
Green lamp ON but the conveyor does not run in AUTO	Register 40001 ("DB_CAMERA".DATA1) is not 0	The PC wrote 1. Click Rescan Stack or Start on the PC, or write 0 in the watch table.
Compile error at %DB2.DBW0 or at MB_HOLD_REG	DB still has "Optimized block access" ticked	Untick it (part 3, step 3) and compile again.
Compile error "tag not defined"	Name typed differently in two places	Pick names from the drop-down list instead of typing.
HMI shows #####	No connection to the PLC	Check both IP addresses, cables, and HMI_Connection_1.
Alarm text does not appear but the red lamp is ON	Wrong trigger bit	Use bits 8 and 9 (part 10.3).
The PC cannot connect	FC_TCP_IP not called in OB1, or wrong port	Check part 9 step 3 and LocalPort = 502.

Part 12. New: red-button reverse jog in MANUAL (not in the old program)

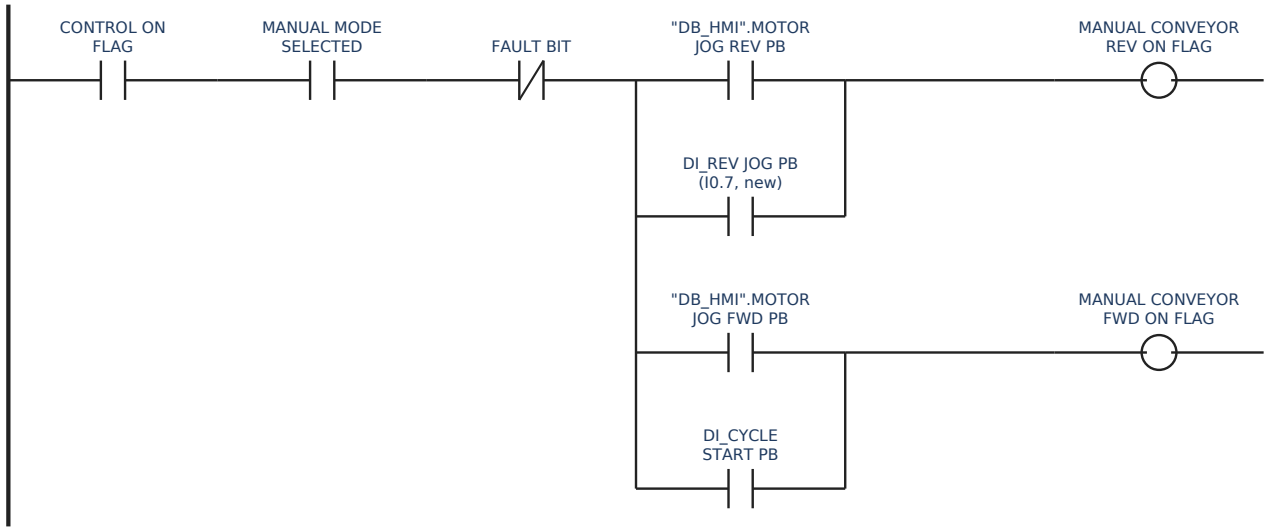
You told me (12 October 2026): in MANUAL mode, holding the **red** button should run the belt in **reverse**, and releasing it should stop the belt, like the HMI REV JOG button. The old program does not do this. Build and test parts 1 - 11 first (an exact copy). Then add this change.

Way	Wiring	Program	If a wire breaks
A (recommended)	Clip one extra **NO** contact block onto the red button. Wire: +24 V - new NO contact - spare input **I0.7**.	Normally open contact on I0.7	Reverse simply does not work. Safe.
B (not recommended)	No new wire: use the existing NC STOP contact on I0.3.	Normally closed contact on I0.3	A broken or loose STOP wire looks like "red button held": in MANUAL the belt would run in **reverse by itself**.

Steps for way A

1. Panel (power OFF): fit a NO contact block on the red button. Wire +24 V to one side, and the other side to PLC input **I0.7**.
2. Tag table **DI**: rename **DI_SPARE I0.7** to **DI_REV JOG PB** (address %I0.7 stays).
3. FC_MANUAL network 1: below the contact **"DB_HMI".MOTOR JOG REV PB** add a parallel branch with a normally open contact **DI_REV JOG PB**, joined again before the coil MANUAL CONVEYOR REV ON FLAG (drawing below).
4. Interlock, so forward and reverse can never be ON together: FC_OUTPUTS network 1, add a **normally closed** contact **DO_CONVEYOR REV ON** just before the coil; network 2, add a normally closed contact **DO_CONVEYOR FWD ON** just before the coil.
5. HMI, INPUT STATUS screen: change the description text of I0.7 from SPARE to **REV JOG PB** (optional).
6. Compile (Errors: 0), download, and test as in the table below.

FC_MANUAL - Network 1 after the change



#	Do this	You must see
1	MANUAL. Hold the red button, then release it	Q0.1 ON while held, OFF when released
2	MANUAL. Hold green and red together	Only one of Q0.0 / Q0.1 ON (forward wins)
3	AUTO, cycle running. Press red	The cycle stops, as before. The belt does not reverse

#	Do this	You must see
4	MANUAL. Disconnect the I0.7 wire, hold red	Nothing moves (way A is safe)

Keep this change out of the exact-copy project until the copy is tested. Save the project first under a new name (Project > Save as) so you can go back to the exact copy at any time.

Appendix A. What the switched-off part of FC_AUTO was for

The old program was first written for another way of working: the sensor was on PLC input I0.0. After the sensor saw a box, the PLC waited STOP TIME seconds (set on the HMI), stopped the belt, wrote 1 to register 40001 as a signal to the PC, and waited until 0 came back (CAMERA DONE) before running on. The HMI showed WAITING FOR CAMERA ACK (status 30) meanwhile.

When the sensor was moved to the camera, the author did not delete this cycle. A **normally closed AlwaysTRUE** contact was put at its start (FC_AUTO network 3, third row) so that it can never become active, and an **AlwaysTRUE** branch was put parallel to the NOT OFF2 contact in the first row, so that it can never stop the belt. That is why network 3 looks complicated but works like a simple rule: **belt runs while ACTIVE and register 40001 = 0**.

Do not remove the normally closed AlwaysTRUE contact. Without it the old cycle would wake up as soon as something is wired to I0.0.